

Deep Learning Metaparameters Cheatsheet

Learning Rate & Optimization

Learning Rate

- **Standard Value:** `0.001` ($1e-3$)
- **Common Range:** `1e-5` to `1e-1`
- **Description:** Controls how much to change weights during training
- **Guidelines:**
 - Start with `0.001` for Adam optimizer
 - Use `0.01` for SGD
 - Too high → divergence, too low → slow convergence

Learning Rate Schedules

- **Step Decay:** Reduce by factor of 0.1 every 10-30 epochs
- **Exponential Decay:** `lr = initial_lr * (decay_rate ^ epoch)`
- **Cosine Annealing:** Smoothly decrease to 0 following cosine curve
- **ReduceLROnPlateau:** Reduce by 0.5-0.1 when validation loss plateaus

Optimizers & Their Settings

Adam (Most Common)

- **Learning Rate:** `0.001`
- **Beta1:** `0.9` (momentum term)
- **Beta2:** `0.999` (squared gradient momentum)
- **Epsilon:** `1e-8`
- **Weight Decay:** `1e-4` to `1e-2`

SGD with Momentum

- **Learning Rate:** `0.01` to `0.1`
- **Momentum:** `0.9`
- **Weight Decay:** `1e-4`

AdamW (Adam + Weight Decay)

- **Learning Rate:** 0.001
- **Weight Decay:** 0.01 to 0.1
- **Other params:** Same as Adam

RMSprop

- **Learning Rate:** 0.001
- **Alpha:** 0.99 (decay rate)
- **Epsilon:** 1e-8
- **Momentum:** 0.0 (typically not used)

Adagrad

- **Learning Rate:** 0.01
 - **Epsilon:** 1e-10
 - **Initial Accumulator:** 0.1
-

Network Architecture

Hidden Layers

- **Standard:** 2-4 layers for simple tasks
- **Deep Networks:** 10-100+ layers
- **Rule of thumb:** Start with 2-3 layers, add more if underfitting

Hidden Units per Layer

- **Small datasets (<1K samples):** 32-128 units
- **Medium datasets (1K-100K):** 128-512 units
- **Large datasets (>100K):** 512-2048 units
- **Common sizes:** Powers of 2 (32, 64, 128, 256, 512, 1024, 2048)

Activation Functions

- **Hidden layers:** ReLU (standard choice)
- **Output layer:**

- Classification: **Softmax** (multi-class), **Sigmoid** (binary)
- Regression: **Linear** (no activation)
- **Alternatives:**
 - **LeakyReLU** ($\alpha=0.01$)
 - **ELU** ($\alpha=1.0$)
 - **Swish** ($\beta=1.0$)
 - **GELU** (approximation=tanh)

Weight Initialization

- **Xavier/Glorot:** For tanh, sigmoid activations
 - **He:** For ReLU activations (standard)
 - **LeCun:** For SELU activations
 - **Orthogonal:** For RNNs
-

Regularization

Dropout

- **Standard Value:** 0.2 to 0.5
- **Dense layers:** 0.5
- **Convolutional layers:** 0.2 to 0.3
- **Input layer:** 0.1 to 0.2 (if used)
- **RNN layers:** 0.2 to 0.5

Batch Normalization

- **Momentum:** 0.9 to 0.99
- **Epsilon:** $1e-5$
- **Position:** After linear layer, before activation
- **Alternatives:** LayerNorm, GroupNorm, InstanceNorm

Weight Decay (L2 Regularization)

- **Standard Value:** $1e-4$ to $1e-2$
- **Small datasets:** $1e-2$ to $1e-1$
- **Large datasets:** $1e-4$ to $1e-3$

- **Vision models:** $1e-4$
- **NLP models:** $1e-2$

Early Stopping

- **Patience:** 10–20 epochs
- **Min Delta:** $1e-4$ (minimum change to qualify as improvement)
- **Restore Best Weights:** `True`

Gradient Clipping

- **Clip Value:** 1.0 to 5.0
 - **Clip Norm:** 1.0 (global norm)
 - **Usage:** Essential for RNNs, helpful for very deep networks
-

Training Parameters

Batch Size

- **Standard:** 32 (most common)
- **Small datasets:** 16–32
- **Large datasets:** 64–256
- **Memory constraints:** 8–16
- **GPU optimization:** 128–512
- **Rule:** Larger batches = more stable gradients, smaller = better generalization

Epochs

- **Start with:** 50–100 epochs
- **Small datasets:** 100–500 epochs
- **Large datasets:** 10–50 epochs
- **Transfer learning:** 5–20 epochs
- **Use early stopping to avoid overfitting**

Validation Split

- **Standard:** 20% (0.2)

- **Small datasets:** 10-15%
- **Large datasets:** 10-20%
- **Time series:** Use temporal split

Cross Validation

- **K-Fold:** 5 or 10 folds
 - **Stratified:** For imbalanced datasets
 - **Time Series:** TimeSeriesSplit
-

Convolutional Neural Networks (CNNs)

Convolutional Layers

- **Kernel Size:** 3x3 (most common), 5x5 , 7x7 , 1x1
- **Stride:** 1 (standard), 2 for downsampling
- **Padding:** 'same' to preserve spatial dimensions
- **Filters:** Start with 32-64 , double after each pooling layer
- **Dilation:** 1 (standard), 2-4 for dilated convolutions

Pooling Layers

- **Pool Size:** 2x2
- **Stride:** 2
- **Type:** MaxPooling (standard), AveragePooling , GlobalAveragePooling

Common CNN Architecture Patterns

Classic CNN

Conv(32) → Conv(32) → MaxPool → Conv(64) → Conv(64) → MaxPool →
Conv(128) → Conv(128) → MaxPool → Flatten → Dense(512) → Dense(num_classes)

ResNet Block

x → Conv → BN → ReLU → Conv → BN → Add(x) → ReLU

Inception Block

Input → [1x1, 3x3, 5x5, MaxPool] → Concatenate

Data Augmentation (Images)

- **Rotation:** $\pm 15-30$ degrees
 - **Zoom:** 0.8-1.2
 - **Horizontal Flip:** 50% probability
 - **Width/Height Shift:** $\pm 10-20\%$
 - **Brightness:** $\pm 20\%$
 - **Contrast:** $\pm 20\%$
 - **Shear:** ± 15 degrees
-

Recurrent Neural Networks (RNNs)

LSTM/GRU

- **Hidden Units:** 128-512
- **Layers:** 1-3 (start with 1)
- **Dropout:** 0.2-0.5 between layers
- **Recurrent Dropout:** 0.2-0.3
- **Return Sequences:** `True` for stacked RNNs, `False` for final layer
- **Bidirectional:** Often improves performance

Sequence Parameters

- **Sequence Length:** 50-200 timesteps
- **Overlap:** 10-50% for sliding windows
- **Padding:** `'post'` or `'pre'`
- **Truncation:** `'post'` or `'pre'`

LSTM Specific

- **Forget Gate Bias:** 1.0 (helps with vanishing gradients)
 - **Cell State Initialization:** Zeros
 - **Hidden State Initialization:** Zeros
-

Transformer Networks

Attention Mechanism

- **Number of Heads:** 8 or 12
- **Key/Query/Value Dimension:** 64 per head
- **Dropout:** 0.1

Position Encoding

- **Max Length:** 512 or 1024
- **Learned vs Sinusoidal:** Learned for most tasks

Feed Forward

- **Hidden Dimension:** 4x attention dimension
 - **Activation:** GELU or ReLU
-

Data Preprocessing

Normalization

- **Images:**
 - Divide by 255 (0-1 range)
 - ImageNet: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]
- **Numerical:**
 - StandardScaler ($\mu=0$, $\sigma=1$)
 - MinMaxScaler (0-1 range)
 - RobustScaler (median-based)

Text Preprocessing

- **Vocabulary Size:** 10K-50K tokens
- **Max Sequence Length:** 128-512 tokens
- **Tokenization:** WordPiece, BPE, SentencePiece
- **Padding Token:** <PAD> or 0
- **Unknown Token:** <UNK> or 1

Feature Scaling

- **Neural Networks:** Always scale inputs
 - **Target Scaling:** For regression, scale targets too
 - **Categorical:** One-hot or embedding layers
-

Task-Specific Guidelines

Image Classification

- **Architecture:** CNN, ResNet, EfficientNet, Vision Transformer
- **Batch Size:** 32–128
- **Learning Rate:** 0.001 (Adam), 0.01 (SGD)
- **Augmentation:** Mandatory for small datasets
- **Transfer Learning:** Use pre-trained ImageNet models

Object Detection

- **Architecture:** YOLO, R-CNN, SSD
- **Batch Size:** 16–32 (due to memory)
- **Learning Rate:** 0.001–0.01
- **NMS Threshold:** 0.5
- **Confidence Threshold:** 0.5

Natural Language Processing

- **Architecture:** Transformer, LSTM, GRU
- **Batch Size:** 16–64
- **Learning Rate:** 2e-5 to 5e-5 (for pre-trained models)
- **Sequence Length:** 128–512 tokens
- **Warmup Steps:** 500–2000

Regression Tasks

- **Output Activation:** Linear (none)
- **Loss Function:** MSE, MAE, Huber
- **Metrics:** MAE, RMSE, R², MAPE
- **Normalization:** Always normalize targets
- **Output Scaling:** Denormalize predictions

Binary Classification

- **Output Activation:** Sigmoid
- **Loss Function:** Binary Crossentropy
- **Threshold:** 0.5 (adjust based on precision/recall needs)
- **Class Weights:** For imbalanced datasets

Multi-class Classification

- **Output Activation:** Softmax
- **Loss Function:** Categorical Crossentropy
- **Label Smoothing:** 0.1 (helps with overconfidence)

Time Series Forecasting

- **Window Size:** 24-168 (depending on frequency)
 - **Horizon:** 1-24 steps ahead
 - **Features:** Lags, rolling statistics, seasonality
 - **Validation:** Time-based split (no shuffling)
-

Loss Functions & Metrics

Classification

- **Binary:** Binary Crossentropy, Focal Loss
- **Multi-class:** Categorical Crossentropy, Sparse Categorical
- **Multi-label:** Binary Crossentropy (sigmoid)

Regression

- **MSE:** Good for normally distributed errors
- **MAE:** Robust to outliers
- **Huber:** Combines MSE and MAE (delta=1.0)
- **LogCosh:** Smooth approximation of MAE

Custom Loss Functions

- **Focal Loss:** For imbalanced classification (alpha=0.25, gamma=2.0)
- **Dice Loss:** For segmentation tasks

- **IoU Loss:** For object detection
-

Debugging & Tuning Tips

If Training Loss is High (Underfitting)

- Increase model capacity (more layers/units)
- Decrease learning rate
- Train for more epochs
- Check data preprocessing
- Reduce regularization
- Check loss function implementation

If Validation Loss > Training Loss (Overfitting)

- Add dropout ($0.3-0.5$)
- Increase weight decay ($1e-3$ to $1e-2$)
- Reduce model capacity
- Add early stopping
- Get more data
- Increase data augmentation

If Both Losses are High (Underfitting)

- Increase model capacity
- Decrease regularization
- Increase learning rate
- Train longer
- Check feature engineering
- Verify data quality

If Training is Unstable

- Reduce learning rate by 10x
- Use gradient clipping ($1.0-5.0$)
- Check for data leakage
- Use batch normalization
- Check for NaN values in data
- Use learning rate warmup

If Model Not Learning (Loss Stuck)

- Check learning rate (try 10x higher/lower)
 - Verify loss function
 - Check data loading
 - Ensure labels are correct
 - Try different initialization
 - Check for dying ReLU (use LeakyReLU)
-

Monitoring & Evaluation

Training Metrics to Track

- **Loss:** Training and validation loss curves
- **Accuracy:** For classification tasks
- **Learning Rate:** Current learning rate
- **Gradient Norm:** Should be ~1.0, not too small/large
- **Weight Histograms:** Check for dead neurons

Validation Strategy

- **Hold-out:** 80/20 or 70/30 split
- **K-Fold CV:** 5 or 10 folds
- **Stratified:** For imbalanced data
- **Time Series:** Temporal validation

Model Selection Criteria

- **Validation Performance:** Primary metric
 - **Generalization Gap:** Val loss - Train loss
 - **Training Time:** Efficiency considerations
 - **Model Size:** Memory/inference constraints
-

Quick Start Templates

Basic Neural Network

```
model_config = {  
    'learning_rate': 0.001,  
    'batch_size': 32,  
    'epochs': 100,  
    'validation_split': 0.2,  
    'optimizer': 'adam',  
    'dropout': 0.3,  
    'weight_decay': 1e-4,  
    'early_stopping_patience': 10  
}
```

CNN Configuration

```
cnn_config = {  
    'learning_rate': 0.001,  
    'batch_size': 64,  
    'epochs': 50,  
    'optimizer': 'adam',  
    'filters': [32, 64, 128],  
    'kernel_size': 3,  
    'pool_size': 2,  
    'dropout': 0.25,  
    'dense_units': 512,  
    'dense_dropout': 0.5  
}
```

RNN Configuration

```
rnn_config = {  
    'learning_rate': 0.001,  
    'batch_size': 32,  
    'epochs': 100,  
    'optimizer': 'adam',  
    'lstm_units': 128,  
    'lstm_layers': 2,  
    'dropout': 0.3,  
    'recurrent_dropout': 0.2,  
    'sequence_length': 100  
}
```

Transfer Learning

```
transfer_config = {  
    'learning_rate': 1e-4, # Lower for pre-trained  
    'batch_size': 16,      # Smaller due to larger models  
    'epochs': 20,          # Fewer epochs needed  
    'optimizer': 'adam',  
    'freeze_base': True,   # Initially freeze base  
    'fine_tune_epochs': 10, # Unfreeze and fine-tune  
    'fine_tune_lr': 1e-5   # Even lower for fine-tuning  
}
```

Advanced Techniques

Learning Rate Scheduling

- **Warm Restarts:** Cosine annealing with restarts
- **Cyclic LR:** Cycle between min/max learning rates
- **One Cycle:** Increase then decrease learning rate
- **Polynomial Decay:** Gradual polynomial decrease

Advanced Regularization

- **Mixup:** Mix samples and labels ($\alpha=0.2$)
- **CutMix:** Cut and mix image patches
- **DropBlock:** Structured dropout for CNNs
- **Spectral Normalization:** Stabilize training

Optimization Tricks

- **Gradient Accumulation:** Simulate larger batches
 - **Mixed Precision:** FP16 training for speed
 - **Model Averaging:** Average multiple checkpoints
 - **Knowledge Distillation:** Transfer from teacher model
-

Hardware Considerations

GPU Memory Optimization

- **Batch Size:** Reduce if OOM (Out of Memory)
- **Model Parallelism:** Split large models
- **Gradient Checkpointing:** Trade compute for memory
- **Mixed Precision:** Use FP16 to save memory

Training Speed

- **Data Loading:** Use multiple workers
 - **Caching:** Cache preprocessed data
 - **Profiling:** Identify bottlenecks
 - **Distributed Training:** Multi-GPU setup
-

Hyperparameter Tuning

Search Strategies

- **Grid Search:** Exhaustive but expensive
- **Random Search:** More efficient than grid
- **Bayesian Optimization:** Smart search
- **Hyperband:** Early stopping + random search

Important Parameters to Tune (in order)

1. **Learning Rate** (most important)
2. **Batch Size**
3. **Network Architecture** (layers, units)
4. **Regularization** (dropout, weight decay)
5. **Optimizer** and its parameters

Tuning Ranges

- **Learning Rate:** $[1e-5, 1e-1]$ (log scale)
 - **Batch Size:** $[8, 16, 32, 64, 128, 256]$
 - **Dropout:** $[0.0, 0.1, 0.2, 0.3, 0.4, 0.5]$
 - **Weight Decay:** $[0, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2]$
-

Adjustment Strategy

1. **Start with defaults** above
 2. **Monitor training/validation curves**
 3. **Adjust one parameter at a time**
 4. **Use validation performance** to guide changes
 5. **Document what works** for your specific domain
 6. **Consider computational budget**
 7. **Automate hyperparameter search** when possible
-

*Remember: These are starting points. Always validate choices with your specific dataset and problem!
The "best" hyperparameters depend heavily on your data, task, and computational constraints.*